

Aulão de Banco de Dados

Agrupar e agregar — pensar em conjuntos

Tópicos Especiais II · ENADE · 2026/2

Prof. M.Sc. Howard Cruz Roatti

A pergunta de hoje

Você já escreve `SELECT`, `WHERE` e `JOIN`. Aí chega esta pergunta simples:



"Temos uma tabela com **uma venda por linha**. Quanto **cada loja** faturou?"

- A consulta **linha a linha** não responde: ela devolve **as vendas**, não **o total por loja**.
- Falta uma ferramenta para **resumir muitos registros em poucos**.

 Hoje a gente monta essa ferramenta do zero: `GROUP BY`, funções de agregação, `HAVING` e `ORDER BY`.
No fim, você resolve uma questão de ENADE sem esforço.

A virada de chave

Jeito procedural (de quem programa)

Percorrer **uma linha de cada vez** e ir somando numa variável.

```
for venda in vendas: total[loja] += venda
```

Jeito do SQL (declarativo)

Você **não percorre** nada. Descreve **grupos** e o que quer de **cada grupo**.

"Para **cada loja**, a **soma** dos valores."

💡 A ideia-mãe: a tabela é um **monte de linhas** → você as separa em **baldes** (grupos) → e pede **um número por balde**. Pensar em **conjuntos**, não em laços.

Nossa tabela de exemplo — VENDAS

loja	vendedor	valor
Norte	Ana	100
Norte	Bia	50
Sul	Caio	200
Sul	Duda	80
Leste	Edu	30

Uma venda por linha.

Cinco linhas, três lojas.

Vamos responder, passo a passo:

1. quanto faturou **no total**?
2. quanto faturou **cada loja**?
3. e **só** as lojas grandes?
4. da **maior** para a menor?

Passo 1 — resumir a tabela inteira

Uma **função de agregação** engole **muitas linhas** e devolve **uma**:

```
SELECT SUM(valor), COUNT(*), AVG(valor) FROM VENDAS;
```

SUM(valor)	COUNT(*)	AVG(valor)
460	5	92

💡 Sem GROUP BY, a tabela toda é **um único grupo** → uma linha de resposta. SUM soma, COUNT conta, AVG tira a média, MIN / MAX pegam extremos.

Passo 2 — GROUP BY: um resumo por grupo

GROUP BY loja separa as linhas em **balde**s e aplica a função em **cada um**:

Entram (agrupadas por cor)

loja	valor
Norte	100
Norte	50
Sul	200
Sul	80
Leste	30

Saem — GROUP BY loja + SUM

loja	SUM(valor)
Norte	150
Sul	280
Leste	30



Cada **cor** vira **uma linha**. Cinco linhas entraram, três saíram — uma por grupo.

A regra de ouro (ela nasce aqui)

No `SELECT`, **cada coluna** ou está no `GROUP BY`, ou está **dentro de uma função** de agregação.

```
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja; --  Loja agrupa, valor agrega
```

E se eu jogar `valor` no `GROUP BY` também?

```
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja, valor; -- 
```

 Agora cada **par (loja, valor)** é um grupo → volta a **uma linha por valor**, e a soma vira o próprio valor. A totalização **some** — e o banco aceita numa boa. Por isso a regra: **agrupa-se pelo que NÃO está agregado**.

Vote no ar 🖐️ — qual totaliza por loja?

```
-- Consulta 1
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja;
-- Consulta 2
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja, valor;
```

🖐️ **1 ou 2?** Pensem no que vira "um balde". Vamos rodar as duas na tela.

- **Consulta 1** → 3 linhas (Norte 150, Sul 280, Leste 30). ✅
- **Consulta 2** → 5 linhas, cada valor sozinho. ❌ (o balde virou "loja + valor")

Passo 3 — WHERE × HAVING

Os dois filtram — mas em **momentos diferentes**:

WHERE → filtra **LINHAS**

antes de agrupar.

```
WHERE valor >= 50
```

(descarta a venda de 30 **antes** da soma)

HAVING → filtra **GRUPOS**

depois de agrupar.

```
HAVING SUM(valor) > 100
```

(descarta a loja cujo **total** ≤ 100)

```
SELECT loja, SUM(valor) FROM VENDAS
GROUP BY loja
HAVING SUM(valor) > 100;      -- só Norte (150) e Sul (280); Leste (30) cai
```

💡 Regra prática: condição sobre **coluna** → WHERE ; condição sobre **resultado de agregação** (SUM , COUNT ...) → HAVING .

Passo 4 — ORDER BY pela agregação

Dá para ordenar **pelo número que você calculou**:

```
SELECT loja, SUM(valor) AS total FROM VENDAS  
GROUP BY loja  
ORDER BY total DESC;           -- Sul 280, Norte 150, Leste 30
```

loja	total
Sul	280
Norte	150
Leste	30

💡 ORDER BY SUM(valor) DESC OU ORDER BY total DESC (pelo apelido) — os dois funcionam.

A consulta inteira — ordem de escrita × de execução

```
SELECT  loja, SUM(valor) AS total      -- 5º: escolhe o que aparece
FROM    VENDAS                       -- 1º: de onde vêm as linhas
WHERE   valor >= 50                   -- 2º: filtra LINHAS
GROUP BY loja                         -- 3º: forma os GRUPOS
HAVING  SUM(valor) > 100              -- 4º: filtra GRUPOS
ORDER BY total DESC;                 -- 6º: ordena o resultado
```

📌 Você **escreve** nessa ordem, mas o banco **executa** assim: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. É por isso que o HAVING pode usar SUM e o WHERE não.

Agora vocês 🤝

Em grupos — 8 minutos

Desafios (mesma tabela **VENDAS**)

Escrevam a consulta para cada pergunta:

1. **Quantas vendas** cada vendedor fez? (*dica: `COUNT`*)
2. **Faturamento por loja**, só as lojas que passaram de **R\$ 100**.
3. A **loja campeã** de faturamento (*uma linha só*).

💡 Comecem sempre pela pergunta "**o que é um grupo aqui?**" — é o que vai no `GROUP BY`.

Desafios — gabarito

-- 1) vendas por vendedor

```
SELECT vendedor, COUNT(*) FROM VENDAS GROUP BY vendedor;
```

-- 2) faturamento por loja, acima de 100

```
SELECT loja, SUM(valor) FROM VENDAS  
GROUP BY loja HAVING SUM(valor) > 100;
```

-- 3) Loja campeã

```
SELECT loja, SUM(valor) AS total FROM VENDAS  
GROUP BY loja ORDER BY total DESC  
LIMIT 1;                -- (no Oracle: FETCH FIRST 1 ROW ONLY)
```

💡 Reparem: **o que muda** entre eles é só o `GROUP BY` e o filtro/ordenação — a estrutura é a mesma.

Como isso cai no ENADE

Banco de eleição: `Partido(numero, nome)`, `Votacao(partido, votos, estado)`. Queremos o **total de votos por partido**. Assinale a consulta correta.

```
-- A
SELECT p.nome, SUM(v.votos) FROM Partido p, Votacao v
WHERE v.partido = p.numero GROUP BY p.nome;

-- B
SELECT p.nome, SUM(v.votos) FROM Partido p, Votacao v
WHERE v.partido = p.numero GROUP BY p.nome, v.votos;
```

💡 É a **mesma ideia** de hoje: SUM totaliza e o GROUP BY leva só as colunas **não agregadas**. Agrupar por v.votos (B) quebraria a soma → a correta é **A**.

Bloco 2 · revisão rápida

Junção e modelagem — como conjuntos

Junção é operação de conjunto

- **INNER** — só a **interseção**
- **LEFT** — tudo da **esquerda** + interseção
- **RIGHT** — tudo da **direita** + interseção

Para "**tudo de A e o que casa de B**": `A LEFT JOIN B`.

```
SELECT *  
FROM Pedido p  
LEFT JOIN Cliente c  
  ON p.id_cliente = c.id;
```

💡 Idioma útil: `A LEFT JOIN B ... WHERE b.id IS NULL` = "o que existe em A e **não** em B".

A ponte: cardinalidade vira chave estrangeira

(mín, máx) : o **primeiro** é o mínimo, o **segundo** o máximo.

- $(1,1)$ = obrigatório e único · $(0,1)$ = **opcional** e único · $(0,N)$ / $(1,N)$ = vários.
- No **mínimo 0**, a FK **aceita nulo**; no mínimo 1, a FK é **NOT NULL**.

⚠ Pegadinha clássica: ler o mínimo **0** de $(0,1)$ como se fosse obrigatório. "No máximo uma" **não** é "exatamente uma".

🔗 Aprofundar: decks de BD **Modelagem Conceitual (ER)** e **Lógica (→ tabelas)**, com os exercícios integradores.

Fecho — leve isto na prova

💡 **Agregação em uma frase:** forme os grupos com `GROUP BY`, resuma cada um com uma função (`SUM` / `AVG` / `COUNT`), **agrupe pelo que não está agregado**, filtre grupos com `HAVING` e ordene com `ORDER BY`.

Para revisar por conta — decks de BD:

- **Normalização** (muito recorrente!)
- **Transações e ACID**
- **Concorrência / isolamento**
- **Índices** (Views, SQL e Indexação)
- **Álgebra relacional**
- **NoSQL e CAP**

Bons estudos!

howard.cruz@faesa.br

≡ Índice

Tópicos Especiais II

Aula 2 · Banco de Dados

revisão completa